

企画書 / CASE STUDY

Study Tracker

学習管理 Web アプリケーション

日々の学習（科目・時間・日付・メモ）を記録し、合計時間や科目別の内訳をダッシュボードで可視化する Web アプリ。

Supabase（PostgreSQL）をバックエンドに、追加・一覧・編集・削除（CRUD）を実装し、Vercel で公開しています。

React 18

TypeScript

Vite

Supabase

PostgreSQL

Vercel

SCSS

種別 / TYPE

学習管理 Web アプリ（SPA）

担当範囲

課題発見・設計・DB 設計・実装・公開（すべて個人）

公開 URL

study-tracker-delta-rouge.vercel.app

データ操作

Create / Read / Update / Delete

公開状況

公開済み（Vercel / 継続デプロイ）

REPOSITORY

github.com/hirotonozaki/study-tracker

01 プロジェクト概要

02 制作背景

03 課題設定

04 ターゲットユーザー

05 解決方法

06 機能一覧

07 画面構成

08 システム構成図

09 技術選定理由

10 データベース設計

11 CRUD 実装内容

12 Supabase 採用理由

13 Vercel 採用理由

14 工夫した点

15 苦労した点

16 学んだこと

17 今後の改善案

課題発見 → 設計 → 実装 → 公開

プロジェクト概要

Study Tracker は、日々の学習を「科目・時間・日付・メモ」で記録し、合計時間や科目別の内訳をダッシュボードで振り返れる Web アプリです。React 18 / TypeScript で構築し、バックエンドに **Supabase (PostgreSQL)** を採用。学習記録の**追加・一覧・編集・削除 (CRUD)** を実装しています。

これまでの HTML / CSS / WordPress による Web 制作に加えて、「データベースに繋いでデータを保存・更新する Web アプリ」を 課題発見から公開まで一人で形にした制作物です。

項目	内容
種別	学習管理 Web アプリケーション (SPA)
フロントエンド	React 18 / TypeScript / Vite / React Router / SCSS
バックエンド	Supabase (PostgreSQL / Row Level Security)
データ操作	Create / Read / Update / Delete (<code>study_records</code> テーブル)
ホスティング	Vercel (公開済み・継続デプロイ)
担当範囲	企画 / 画面設計 / DB 設計 / 実装 / デプロイ (すべて個人)

制作背景

プログラミングや Web の学習を続ける中で、日々の学習時間を**手軽に記録し、後から振り返れる仕組み**が欲しいと感じたのが出発点です。メモやノートでは「どの分野にどれだけ時間を使っているか」が見えづらく、自分の学習を客観的に把握できる小さなツールを、自分の手でつくろうと考えました。

あわせて、これまで中心だった HTML / CSS / WordPress の Web 制作に加えて、「**データベースに繋ぎ、データを保存・更新する Web アプリ**」を一から作った経験を、きちんと形にして示したいという狙いもありました。

数あるテーマの中で「学習記録アプリ」を選んだ理由は、題材として過不足がなかったためです。記録の**追加・取得・更新・削除 (CRUD)** と、合計時間・科目別といった**集計**が一通り必要になり、さらに SaaS のような**ダッシュボード UI** も練習できます。「DB を使った Web アプリ開発」を学ぶのに ちょうどよい規模だと判断しました。

この制作で示したいこと

Web 制作だけでなく、**DB を扱う Web アプリの開発 (CRUD・集計・可視化)** を、課題発見から設計・実装・公開まで一人で完結できること。

課題設定

「ユーザーが感じる課題」と「制作を通じて技術的に達成したいこと」の2軸で設定しました。

USER - ユーザーの課題

勉強した内容を、手間をかけずにさっと記録したい。

「どの科目をどれだけやったか」を一目で振り返りたい。

続けるうちに増える記録を、見やすく整理したい。

TECHNICAL - 技術的なゴール

Supabase (PostgreSQL) に対して **CRUD を一通り実装**する。

DB の行・フォーム入力・集計結果を **型で表現**し、壊れにくくする。

データ操作 (API) と画面 (表示) を分け、見通しのよい設計にする。

合計・科目別・日付別の**集計を自前で実装**し、可視化する。

ターゲットユーザー

想定するのは、**独学やスキルアップのために学習を継続している人**です。学習アプリやノートはあるものの、「科目ごとの合計を後から振り返る」ところまでは手が回らない、という層を念頭に置いています。

学

独学で学習を続ける人（例）

PERSONA - INDEPENDENT LEARNER

毎日少しずつ複数科目を勉強する。記録は続けたいが、入力が面倒だと続かない。「今週どれだけやれたか」「どの科目が手薄か」を、開いてすぐ把握できる手軽さを求めている。

入力項目を「科目・時間・日付・メモ」の最小限に絞り、開いた最初の画面で合計と科目別の内訳が見えることを、設計の優先事項にしています。

解決方法

課題に対して、機能と画面で次のように応えました。

課題	アプリでの解決
手間なく記録したい	科目・時間（分）・日付・メモだけの 最小フォーム 。日付は当日を初期値に。
科目ごとの偏りを振り返りたい	ダッシュボードに 科目別の合計時間を横棒バー で表示。
増える記録を整理したい	記録を 日付別にグループ化 して一覧表示。
間違えても直したい	各記録を その場で編集・削除（CRUD） 。削除は確認ダイアログ付き。
全体の進み具合を知りたい	合計時間・記録数・科目数・学習日数を 統計カード で集計表示。

機能一覧

C 学習記録の追加

科目・学習時間（分）・学習日・メモを登録（Create）。

R 一覧表示（日付別）

記録を学習日ごとにまとめ、日付の合計時間とともに表示（Read）。

U 記録の編集

既存の記録を選び、同じフォームで内容を更新（Update）。

D 記録の削除

確認ダイアログのうえで記録を削除（Delete）。

科目別の合計時間

科目ごとの合計を、多い順に横棒バーで可視化。

Σ ダッシュボード集計

合計学習時間・記録数・科目数・学習した日数を統計カードで表示。

✓ 入力バリデーション

科目は必須、学習時間は 1 以上の数値のみ受け付ける。

🔄 状態表示

読み込み中／取得・保存・削除の失敗時にメッセージを表示。

画面構成

サイドバー付きの SaaS 風レイアウトで、画面は**ダッシュボード**と**学習記録**の 2 つ。ルーティングは react-router-dom (HashRouter) で、静的ホスティングでもそのまま動く構成です。

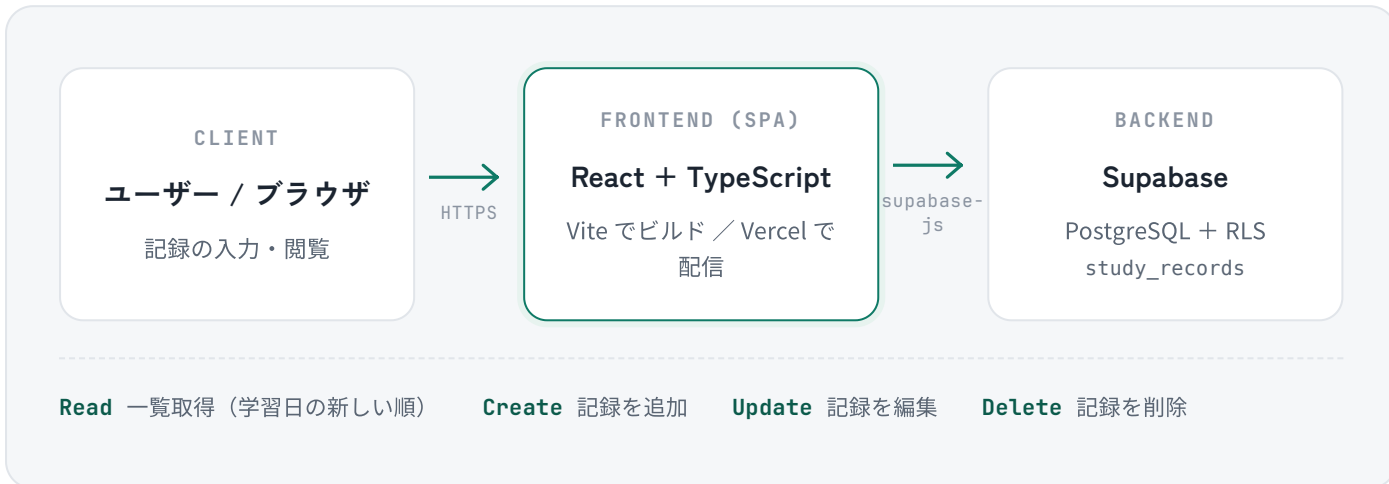


画面	パス	役割
ダッシュボード	/	学習データの集計を一目で把握 (Read+集計の表示)。統計カード／科目別バー／最近の記録。
学習記録	/records	記録の追加・編集・削除 (CRUD の操作画面)。左に入力フォーム、右に日付別の一覧。

レスポンシブ対応：880px 以下でサイドバーが上部の横並びに変化し、グリッドは 1 カラムに切り替わります。

システム構成図

フロントエンドの SPA を Vercel から配信し、ブラウザ上の @supabase/supabase-js クライアントを通じて Supabase（PostgreSQL）と直接やり取りします。自前のサーバーは持たないシンプルな構成です。



画面は `api.ts`（Supabase 呼び出し）→ `useStudyRecords`（状態管理）を経由してデータを扱い、コンポーネントは SQL を意識しません（詳細は 11 章）。

技術選定理由

技術	採用理由
React 18 / TypeScript	状態に応じて画面が更新される宣言的 UI と、型による安全性を両立できるため。
Vite	開発サーバーが軽快で、学習中の試行錯誤（保存→即反映）がしやすいため。
Supabase (PostgreSQL)	PostgreSQL をすぐ使え、JS クライアントから CRUD を素直に書けるため（詳細は 12 章）。
React Router	ダッシュボードと記録画面を分けるルーティングのため。静的配信向けに <code>HashRouter</code> を採用。
SCSS	変数・ネストでデザインを見通しよく管理するため（デザイントークン＋BEM ライク命名）。

データベース設計

学習記録は「ある日・ある科目を何分やったか」が1件の情報。これを `study_records` テーブル 1 つで表現しました。

study_records		TABLE
id PK	uuid	主キー (<code>gen_random_uuid()</code>)
subject	text	科目名 (not null)
minutes	integer	学習時間・分 (<code>check > 0</code>)
studied_on	date	学習日 (default 当日)
memo	text	メモ (任意・null 可)
created_at	timestampz	作成日時 (default <code>now()</code>)

■ 設計上の判断

時間は分 (integer) で保持

「1.5 時間」のような小数を持たず、整数で扱うことで計算・集計が単純に。表示時に「1h 30m」へ整形。

科目はテーブルを分けない

規模が小さく、科目別集計は `subject` でのグループ化で足りるため非正規化。正規化は今後の改善案 (17 章)。

集計はアプリ側で算出

合計／科目別／日付別を `utils.ts` の純粋関数で計算し、ダッシュボードに渡す。

インデックス

並び替え・集計でよく使う `studied_on` (降順) と `subject` に付与。

本番化メモ - RLS と認証について

現在はログイン無しの**単一ユーザー想定**のデモのため、Row Level Security を有効化したうえで `anon` に CRUD を許可しています。実運用では Supabase Auth を導入し、`user_id` 列を追加して「`auth.uid()` = `user_id` の行だけ操作可」へ置き換える前提です (17 章)。今回は「DB に繋いで CRUD する」ことを示すため、あえてシンプルにしています。

CRUD 実装内容

Supabase への読み書きは `api.ts` に集約し、画面からは関数を呼ぶだけ（SQL を意識しない）構成です。取得後はカスタムフック `useStudyRecords` がローカル state を持ち、操作のたびに全件を取り直さず **該当分だけ更新して即時反映**します。

操作	関数 (API.TS)	画面への反映
Create	<code>createRecord()</code>	返ってきた 1 件を一覧の 先頭に追加 。
Read	<code>fetchRecords()</code>	学習日の新しい順で取得し、state を初期化。
Update	<code>updateRecord(id)</code>	該当 <code>id</code> の 1 件を 差し替え 。
Delete	<code>deleteRecord(id)</code>	該当 <code>id</code> を state から 除外 。

■ 追加（Create）の流れ

- 1 フォームに入力して送信する。
- 2 入力を**バリデーション**（科目が空でないか／時間が 1 以上か）。
- 3 `api.ts` の `createRecord()` が Supabase に insert する。
- 4 返ってきた 1 件を**ローカル state の先頭に追加**して即時反映（再取得しない）。
- 5 失敗時は `try / catch` で**エラーメッセージ**を表示する。

```
// api.ts - Create (追加)
export async function createRecord(input: StudyRecordInput): Promise<StudyRecord> {
  const { data, error } = await supabase
    .from('study_records')
    .insert({
      subject: input.subject,
      minutes: input.minutes,
      studied_on: input.studied_on,
      memo: input.memo.trim() === '' ? null : input.memo,
    })
    .select()
    .single();

  if (error) throw error;
  return data;
}
```

12 WHY SUPABASE

Supabase 採用理由

DATABASE

PostgreSQL をすぐ使える

サーバーを自前で立てずに、本格的なリレーショナル DB を最初から使える。学習用途でも実務に近い形を体験できる。

CLIENT

JS から CRUD が書きやすい

@supabase/supabase-js で insert / select / update / delete を素直に記述でき、フロント側の実装に集中できる。

SECURITY

RLS と認証の道筋がある

Row Level Security と Supabase Auth が用意されており、デモ → 本番（ユーザー別データ）への発展経路が明確。

OPS

運用がシンプル

管理するサーバーがなく、個人で「設計 → 実装 → 公開」まで完結させやすい。

Vercel 採用理由

DEPLOY

GitHub から自動デプロイ

リポジトリと連携し、push するだけでビルド・公開まで自動化。継続的に更新しやすい。

CONFIG

環境変数の管理が容易

Supabase の URL / anon key を環境変数として安全に設定でき、鍵をコードに含めずに済む。

HOSTING

SPA の静的配信に最適

ビルド済みの静的アセットを高速に配信。
HashRouter 採用でルーティングもそのまま動作。

HTTPS

公開までが速い

HTTPS で即時公開でき、ポートフォリオとして URL を共有しやすい。

工夫した点

工夫	内容
データ操作の分離	Supabase 呼び出しを <code>api.ts</code> に閉じ込め、状態同期を <code>useStudyRecords</code> に集約。画面は「表示」に専念し、後から機能を足しても壊れにくい構成に。
型で壊れにくく	DB の行 (<code>StudyRecord</code>) ・フォーム入力 (<code>StudyRecordInput</code>) ・集計結果を、それぞれ別の型として定義。
集計を純粋関数に	科目別・日付別・合計の集計を <code>utils.ts</code> の純粋関数として切り出し、見通しよく・テストしやすく。
CRUD 後の即時反映	操作のたびに全件を再取得せず、返回值や <code>id</code> を使ってローカル state だけ更新し、体感を軽くした。
入力・状態のケア	科目必須・時間 1 以上のバリデーション、読み込み中／エラー表示、削除前の確認ダイアログを用意。

苦勞した点

つまずき	解決方法
環境変数と接続	URL / anon key が未設定だと動かず最初につまずいた。→ クライアント生成時に未設定を検知し、原因が分かるエラーを出すようにした。
RLS で操作できない	RLS を有効化したら最初は読み書きできなかった。→ 仕組みを理解し、デモ用に <code>anon</code> を許可するポリシーを設定（本番は Auth 前提と整理）。
状態の同期	更新・削除のたびに全件再取得すると重い。→ 返り値や <code>id</code> を使い、ローカル state だけを更新する形にした。

学んだこと

01 DB を使った CRUD を一通り実装できた

取得・追加・編集・削除という Web アプリの基礎を、フロントから DB まで通して理解した。

02 データ層と画面の分離の効果

API・状態管理・表示を分けると、機能追加や修正がしやすくなることを体感した。

03 型を起点に設計する感覚

DB の行・入力・集計を型で表すと、実装中の取り違えに早く気づけた。

04 RLS・認証の入り口

RLS でアクセスを制御する考え方と、本番では認証が必要になる理由を理解した。

05 公開までやり切る経験

環境変数を設定し、実際に動く URL として公開するまでの一連を経験した。

今後の改善案

背伸びをせず、現状の延長で着実に取り組める範囲で整理しました。

改善案	狙い
subjects テーブルへの正規化	科目を別テーブルに切り出し、外部キー・JOIN を扱えるようにする。
Supabase Auth の導入	ログインを追加し、 <code>user_id</code> 単位の RLS でユーザーごとの記録に対応（本番仕様）。
推移グラフ・目標時間	週次・月次の推移を可視化し、目標時間を設定できるようにする。
テストの追加	Vitest + React Testing Library で、集計ロジックや主要フローを検証する。

この制作で活かせること

DB を使った CRUD・集計・可視化（ダッシュボード）の流れは、**SaaS プロダクトの管理画面**に通じます。ここを起点に、フロントから DB を扱う側の理解を深めていきたいと考えています。

Study Tracker

React・TypeScript・Supabase・Vercel / DB を使った CRUD Web アプリ

LINKS

- ▶ study-tracker-delta-rouge.vercel.app
- ▶ github.com/hirotonozaki/study-tracker